

Achado → mitigação, ferramenta por ferramenta

Cada card é uma ferramenta rodada de verdade contra código/dependências/imagem/app reais — a categoria que ela cobre (sigla explicada), o problema que apareceu na prática, as ações tomadas (e as que ainda faltam), e se o item aparece explicitamente no anúncio da vaga de Staff DevSecOps da Analog Devices (a mesma que motivou a call de triagem com a Casey Galvin, 22/09).

01 Gitleaks

SECRETS

Secrets scanning — não é sigla: é a prática de detectar segredos (chaves, senhas, tokens) commitados no código ou no histórico do git antes que alguém os explore.

COMO FUNCIONA

CLI em Go que varre o `git log` commit por commit (não só o working tree), aplicando regex + detecção de entropia contra padrões conhecidos (chave AWS, token Stripe/GitHub etc.) e regras próprias em TOML.

COMO EU USO

`gitleaks detect --source . --no-banner` contra o repo inteiro. Plano: virar pre-commit hook, hoje ainda roda manual.

RESOLVE

Credencial hardcoded commitada no git — some do arquivo, mas não do histórico.

ACHADO REAL

0 leaks no repo real. Demo provocada (repo git descartável): removi a chave num commit "fix" e o Gitleaks ainda achou, apontando o commit antigo — `git show` confirmou que a chave inteira ainda existe lá.

AÇÕES

IMPLEMENTADA

Rodei o Gitleaks contra o histórico real e provoquei um leak num repo descartável pra confirmar o comportamento.

→ **0 leaks** reais, causa raiz explicada

PENDENTE

Instalar como pre-commit hook no repo (hoje só roda manual) e rotacionar uma credencial de verdade após um leak.

VS. VAGA DA CASEY (ANALOG DEVICES)

IMPLÍCITA

Não aparece como sigla isolada no anúncio, mas é a base de qualquer pipeline seguro — está implícita em "SSDLC" e em secrets management dentro de AWS (IAM/KMS), que a vaga cita.

leak provocado → **detectado + causa raiz explicada**

SAST = Static Application Security Testing — analisa o código-fonte sem executá-lo, procurando padrões conhecidos de vulnerabilidade.

COMO FUNCIONA

Parseia o código numa AST (árvore de sintaxe) e casa "patterns" escritos numa DSL parecida com o próprio código-fonte contra essa árvore — é estrutural, não regex de texto, mas ainda depende de existir um pattern pro bug exato.

COMO EU USO

`semgrep scan --config auto .` no dia a dia. Quando acho um gap, escrevo regra própria em YAML e rodo `--config auto --config minha-regra.yaml` junto.

RESOLVE

Bug de segurança no código que você mesmo escreveu (injection, hash fraco, secret hardcoded).

ACHADO REAL

Ruleset padrão (`auto`) pegou 3 de 4 bugs plantados — perdeu `os.system("..." + var)` , mesma classe de `subprocess.call(shell=True)` , só com sintaxe diferente.

AÇÕES

IMPLEMENTADA

Escrevi uma regra Semgrep própria em YAML (`pattern: os.system("..." + $VAR)`) fechando exatamente o gap.

→ 3 → **4 findings**

PENDENTE

Aplicar os padrões de fix no código real (sem `shell=True`, hash com `scrypt/bcrypt`, secret em variável de ambiente) — hoje só existem no arquivo de demo.

VS. VAGA DA CASEY (ANALOG DEVICES)

CITADA EXPLICITAMENTE

SAST é uma das siglas do anúncio — um dos gaps centrais que o job-bot mapeou antes da candidatura, ao lado de DAST/SCA/SBOM.

3 findings → **4 findings**

SCA = Software Composition Analysis — audita as dependências de terceiros que o projeto importa, cruzando nome+versão com bancos de CVE conhecida.

COMO FUNCIONA

Lê o manifesto de dependências (`requirements.txt` , lockfile etc.), extrai nome+versão exata de cada pacote, e cruza contra um banco de CVE baixado e cacheado localmente — puro lookup, nunca analisa o código da lib em si.

COMO EU USO

`trivy fs --scanners vuln .` antes de subir uma dependência nova ou como gate de CI. Precisa de versão pinada (`==`) — range não é lido.

RESOLVE

CVE conhecida em biblioteca de terceiros que o projeto importa — não é código seu, mas é seu risco.

ACHADO REAL

`requests==2.6.0` / `pyyaml==5.1` / `urllib3==1.24.1` → 20 CVEs (3 críticas), incluindo RCE no PyYAML via `FullLoader`.

AÇÕES

IMPLEMENTADA Pinei versão corrigida exata (==) de cada lib e reescaneei. → 20 → 0 CVEs

IMPLEMENTADA Descobri que range (>=) faz o Trivy nem escanear — só versão fixa funciona. → virou regra pro time

PENDENTE

Configurar Dependabot/Renovate no repo real pra manter isso automático, e o gate do CI ainda não rodou contra um PR de verdade.

VS. VAGA DA CASEY (ANALOG DEVICES)

CITADA EXPLICITAMENTE

SCA está no anúncio ao lado de SAST/DAST/SBOM — mesmo bloco de gap mapeado pelo job-bot.

20 CVEs → 0 CVEs

Container scanning — não é sigla: escaneia a imagem final (SO base + libs de sistema + app), não só o código-fonte que você escreveu.

COMO FUNCIONA

Mesma engine de CVE do Trivy fs, mas desmonta a imagem Docker camada por camada, lista os pacotes do SO instalados (dpkg/apk/rpm) + os pacotes de linguagem que estão dentro da imagem, e cruza tudo contra o mesmo banco de CVE.

COMO EU USO

`docker build -t app:tag .` seguido de `trivy image --severity CRITICAL,HIGH app:tag`. Comparo antes/depois toda vez que troco base image ou atualizo dependência.

RESOLVE

Vulnerabilidade herdada da imagem base (SO + libs de sistema) — invisível olhando só o código da app.

ACHADO REAL

`python:3.11` completo = 654 CVEs críticas/altas, quase tudo do Debian base (não da app).

AÇÕES

IMPLEMENTADA Troquei a base pra `-slim`. → 654 → **62**

IMPLEMENTADA Atualizei fastapi (puxou starlette sem CVE). → 62 → **59**

IMPLEMENTADA Tentei atualizar pip/setuptools/wheel pra fechar as 2 CVEs restantes do Python.

→ trocou por outras 2, sem ganho — revertido

PENDENTE

Multi-stage build / distroless pra remover perl/util-linux não usados pela app (3 CRITICAL residuais) e agendar rebuild periódico pra pegar patch da base image.

VS. VAGA DA CASEY (ANALOG DEVICES)

IMPLÍCITA

Não vem como termo isolado no anúncio, mas cai dentro do escopo cloud-native que a vaga cita via ECS/EKS — sem container scanning não dá pra rodar CVE-free nesse tipo de ambiente.

654 → **59**

3 CRITICAL residuais no SO (Perl não usado pela app)

SBOM = Software Bill of Materials — inventário formal de tudo que compõe o artefato (componentes, versões, licenças), pra responder rápido quando sai uma CVE nova.

COMO FUNCIONA

Também desmonta imagem/diretório como o Trivy, mas em vez de comparar contra CVE, só cataloga tudo que encontra (pacotes de SO, libs de linguagem, binários) e serializa num formato padrão (CycloneDX ou SPDX) — inventário puro, sem lookup de vulnerabilidade.

COMO EU USO

`syft dir:. -o cyclonedx-json > sbom.json` depois de cada build. Objetivo: publicar como artefato do CI, não gerar manual.

RESOLVE

Sem inventário, uma CVE nova (ex.: Log4Shell) exige re-escanear tudo do zero pra saber se te afeta.

ACHADO REAL

SBOM do repo real = 40 componentes em CycloneDX, incluindo GitHub Actions do CI — não só libs de linguagem.

AÇÕES

IMPLEMENTADA Gerei o SBOM do repo real e do app de lab em CycloneDX. → **40 componentes** catalogados

PENDENTE

Simular uma CVE nova de verdade e consultar o SBOM pra responder em minutos (nunca fiz esse exercício), e confirmar que o job de CI publica o artefato a cada build.

VS. VAGA DA CASEY (ANALOG DEVICES)

CITADA EXPLICITAMENTE — PESO EXTRA

SBOM está no anúncio, e aqui pesa mais que boa prática genérica: a ADI tem linha regulada como SaMD (Software as a Medical Device), e a FDA exige SBOM na submissão de software médico desde 2023 — pra esse produto específico é requisito regulatório, não opcional.

sem inventário → **40 componentes catalogados**

DAST = Dynamic Application Security Testing — ataca a aplicação já rodando, de fora, simulando um atacante real; nunca olha o código-fonte.

COMO FUNCIONA

Sobe um proxy que primeiro navega pela app (spider/crawl) descobrindo rotas, depois manda os requests capturados de volta com variações (headers ausentes, payloads simples) e audita as respostas — tudo via HTTP, de fora.

COMO EU USO

Subo a app com `docker run`, depois `docker run zapproxy/zap-stable zap-baseline.py -t http://host.docker.internal:PORT/` — o "baseline" é o modo rápido e majoritariamente passivo, não-destrutivo.

RESOLVE

Falha que só existe com a app rodando de verdade (headers, config de runtime) — invisível lendo o código-fonte.

ACHADO REAL

`app.py` de 12 linhas, sem nenhum bug de lógica, ainda assim com 3 headers de segurança ausentes em runtime.

AÇÕES

IMPLEMENTADA

Middleware adicionando X-Content-Type-Options, Cross-Origin-Resource-Policy e Cache-Control. → 3 → **1 WARN**

PENDENTE

Configurar scan autenticado (hoje só cobre rota anônima); o WARN residual é efeito esperado do próprio fix — decisão consciente, não pendência.

VS. VAGA DA CASEY (ANALOG DEVICES)

CITADA EXPLICITAMENTE

DAST está no anúncio ao lado de SAST/SCA/SBOM — fecha o quarteto de siglas que motivou o crash course inteiro.

3 WARN → **1 WARN**

residual é efeito esperado do próprio fix

Detalhe completo de cada scan em `notes/devsecops-scan-report.md` · passo a passo pra reproduzir em `notes/devsecops-crash-course.md` · perguntas de entrevista + régua C→B em `notes/devsecops-interview.md` — todos em `devops-bootcamp/`.